



POSLink2 Semi-Integration Setup Guide

04-23-2025

V1.00

Preface

Copyright © 2025 PAX Technology, Inc. All rights reserved.

The Programs (which include both the software and documentation) contain proprietary information; they are provided under a license agreement containing restrictions on use and disclosure and are also protected by copyright, patent, and other intellectual and industrial property laws. Reverse engineering, disassembly, or decompiling of the Programs, except to the extent required to obtain interoperability with other independently created software or as specified by law, is prohibited.

This document is provided for informational purposes only. All features and specifications are subject to change without notice. If there are any problems in the documentation, please report them to PAX in writing. This document is not warranted to be error-free. Except as may be expressly permitted in the license agreement for these Programs, no part of these Programs may be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose.

Security Notice: No part of this publication may be copied, distributed, stored in a retrieval system, translated into any human or computer language, transmitted, in any form or by any means, without the prior written consent of PAX Technology, Inc.

PAX is a registered trademark of PAX Technology Limited in China and/or other countries. All other trademarks or brand names are the properties of their respective holders. PAXBiz, PAXSTORE, POSDK, The PAX Portfolio Manager, and/or other PAX products referenced herein are either trademarks or registered trademarks of PAX Technology, Inc. or its Affiliates. Other product and company names mentioned herein may be trademarks of their respective owners.

The *Bluetooth*® word mark and logos are registered trademarks owned by Bluetooth SIG, Inc., and any use of such marks by PAX Technology, Inc. is under license. Other trademarks and trade names are those of their respective owners.

EMV® is a registered trademark in the U.S. and other countries and an unregistered trademark elsewhere. The EMV trademark is owned by EMVCo, LLC.

Android is a trademark of Google LLC

TECHNICAL SUPPORT

If there is a problem while installing, registering, or operating this product, please make sure to review the documentation. If unable to resolve the issue, please contact PAX.

Monday-Friday 9:00 AM to 1:00 AM
Saturday 9:00 AM to 5:00 PM
Sunday Closed

The level of access to this Service is by the support plan arrangements made between PAX and the Organization. Please consult this support plan for further information about entitlements, including the hours when telephone support is available.

TECHNICAL SUPPORT CONTACT INFORMATION

Phone: (877) 859-0099
Email: support@pax.us
URL: www.pax.us

Revision History

Date	Version	Description
04-23-2025	v1.0	Initial release.

Table of Contents

1. General Information	1
1.1 Document Purpose	1
1.2 Authorized User Permission	1
2. SDK Information	1
2.1 POSLink2	1
3. ECR/POS Setup	2
3.1 .NET POSLink	2
3.1.1 Folder Structure	2
3.1.2 Configure Library Dependencies	3
3.1.3 Setting Up Logs	3
3.1.4 Configuring Communication Settings	3
3.1.5 Performing a Transaction	4
3.2 Java/Android	4
3.2.1 Folder structure	4
3.2.2 configure library dependencies	5
3.2.3 Setting up logs	6
3.2.4 Configuring communication settings	6
3.2.5 Performing a transaction	7
3.3 iOS/MacOS POSLink	8
3.3.1 Folder Structure	8
3.3.2 Configure Library Dependencies	8
3.3.3 Setting up logs	8
3.3.4 Configuring communication settings	8
3.3.5 performing a transaction	9
4. Communication Setting Setup	10
4.1 UART	10
4.2 TCP	10
4.3 HTTP/HTTPS	10
4.4 SSL	11
4.5 Bluetooth	11
4.6 USB	11
4.7 AIDL	11
5. Pax Terminal Setup	12
5.1 Configure ECR Communication Settings via PAXSTORE	12
5.2 Configure ECR Communication Settings via Terminal settings	12
5.2.1 Pax android terminals	12
5.2.2 Traditional Terminals	20
5.2.3 ethernet - tcp/ip, http, https, ssl	21
5.2.4 USB	23
6. HTTPS/SSL Communication Setup Requirement	24
6.1 PEM Files and Requirements	24
6.1.1 serverkey.pem - private key	24
6.1.2 serverca.pem, servercert.pem	24
6.1.3 Verifying that the private key matches a certificate	25
6.1.4 Read certificate information	25
6.2 Load PEM Files to PAX Terminal	27
6.3 Generating a self-signed certificate	28
6.4 Import Self-Signed CA to Trusted Root Certification Authorities (Windows Only)	28
7. PAX Customer Support	29

1. General Information

1.1 Document Purpose

This document provides the detailed description for all user functions, which can be performed on the POS terminal. The document is intended for use by the Software Engineers developing and maintaining the Payment Application design for the US Payment Card Industry, and also for use by customers and Independent Sales Vendors (ISV). The document should be maintained along with the application itself.

We suggest keeping this document handy for future reference.

1.2 Authorized User Permission

Unless explicitly stated otherwise all content included on this document, such as but not limited to text, images, logos, and slogans, is the property of PAX and is protected by U.S. and international copyright laws.

Any other use, including the reproduction, modification, display or distribution, of the content on this system is strictly prohibited. No portion of this system may be reproduced, duplicated, copied, sold, resold, or otherwise, exploited for any purpose.

2. SDK Information

2.1 POSLink2

- Three SDK versions:
 - .NET Standard
 - iOS
 - Java/Android
- Support both Semi-Integration and Full-Integration APIs.
- Support USB, Serial Port, TCP, SSL/TLS, HTTP, HTTPS, and Bluetooth ECR communication types.
- Only POSLink2 Core Android Library supports AIDL (Android Interface Definition Language) ECR communication type. AIDL is for A920, A35, etc. devices (Android terminals).
- The E-series (E700, E800) uses Android version with USB only.

3. ECR/POS Setup

3.1 .NET POSLink

3.1.1 FOLDER STRUCTURE

```

Root Folder
├── Libs
│   └── Contains DLLs required for development
├── ComRegister
│   └── Contains COM registration files and TLB files for COM-based development
├── Doc
│   ├── API Guide
│   │   └── Provides documentation on DLL APIs.
│   └── Reference
│       └── Offers additional programming details beyond the API guide.
├── Samples
│   ├── FlutterDemo
│   │   ├── Code
│   │   │   └── Contains source code for the Flutter demo.
│   │   └── Demo
│   │       └── Includes the compiled POSLink Flutter demo.
│   ├── POSLinkUart
│   │   └── Demonstrates UART communication using DLLs.
│   ├── .NetFrameworkDemo
│   │   └── Contains a simple POSLink demo for .NET Framework
│   └── MAUI
│       ├── G1CommBinding
│       │   └── UART communication sample for Android in MAUI.
│       └── POSLinkMauiDemo
│           └── POSLink MAUI demo application.
    
```

3.1.2 CONFIGURE LIBRARY DEPENDENCIES

1. Locate the necessary DLLs in the **Libs** folder.
2. Copy them to your project directory (e.g., **bin** or **libs**).
3. Add references to the DLLs in your IDE.

3.1.3 SETTING UP LOGS

```
// Create an instance of POSLinkSemi to handle semi-integration operations
POSLinkSemiIntegration.POSLinkSemi poslink =
    POSLinkSemiIntegration.GetPOSLinkSemi();

// Initialize a log setting object to configure logging behavior
POSLinkCore.LogSetting logSetting = new POSLinkCore.LogSetting();

// Enable logging
logSetting.Enable = true;

// Set Log Level to Debug to capture detailed logs
logSetting.Level = POSLinkCore.LogSetting.LogLevel.Debug;

// Set Log retention to 30 days
logSetting.Days = 30;

// Specify the log file path (current directory)
logSetting.FilePath = ".\\\\";

// Apply the log settings to the POSLink instance
poslink.SetLogSetting(logSetting);
```

3.1.4 CONFIGURING COMMUNICATION SETTINGS

```
// Create a new instance of TcpSetting for configuring TCP communication
POSLinkCore.CommSetting.TcpSetting tcpSetting = new
    POSLinkCore.CommSetting.TcpSetting();

// Set the IP address of the device or server to communicate with
tcpSetting.Ip = "127.0.0.1"; // Localhost (change this to the actual device IP if
    needed)

// Set the port number for communication
tcpSetting.Port = 10009; // Ensure this matches the configured listening port of
    the target device

// Set the timeout duration for the connection in milliseconds
tcpSetting.Timeout = 60000; // 60 seconds timeout
```

Note: Since .NET Standard lacks the SerialPort library, when working with the UART CommType, you will need to inherit and override the CustomerCommSetting class. For a reference implementation, please refer to the POSLinkUart sample.

3.1.5 PERFORMING A TRANSACTION

```
// Get a terminal instance using the specified TCP communication settings
POSLinkSemiIntegration.Terminal terminal = poslink.GetTerminal(tcpSetting);

// Create an AmountReq object to set the transaction amount
POSLinkAdmin.Util.AmountReq amountReq = new POSLinkAdmin.Util.AmountReq();
amountReq.TransactionAmount = "100"; // Set transaction amount to 100 (currency
unit depends on config)

// Create a TraceReq object to set the reference number for tracking the
transaction
POSLinkSemiIntegration.Util.TraceReq traceReq = new
POSLinkSemiIntegration.Util.TraceReq();
traceReq.EcrReferenceNumber = "1"; // Assign a reference number for this
transaction

// Create a DoCreditReq object to define the credit sale request
POSLinkSemiIntegration.Transaction.DoCreditReq doCreditReq = new
POSLinkSemiIntegration.Transaction.DoCreditReq();
doCreditReq.TransactionType = POSLinkAdmin.Const.TransactionType.Sale; // Set
transaction type to Sale
doCreditReq.AmountInformation = amountReq; // Attach amount details
doCreditReq.TraceInformation = traceReq; // Attach trace details

// Initialize response and execution result objects
POSLinkSemiIntegration.Transaction.DoCreditRsp doCreditRsp = null;
POSLinkAdmin.ExecutionResult executionResult = null;

// Execute the credit sale transaction and retrieve the response
executionResult = terminal.Transaction.DoCredit(doCreditReq, out doCreditRsp);

// Check if the transaction was successful
if (executionResult.GetErrorCode() == POSLinkAdmin.ExecutionResult.Code.Ok)
{
    // Print the response code from the transaction
    Console.WriteLine(doCreditRsp.ResponseCode);
}
```

Note: When developing Android applications with Xamarin or MAUI, you may encounter an SSL Authentication Failed exception. This issue is discussed in this [Xamarin-Android update](#). To resolve it, disable server certificate validation on Android.

3.2 Java/Android

Note: Many Android reference topics are derived from Java-based source code. This means all Android reference are the same with Java's, except some Android specific functions.

3.2.1 FOLDER STRUCTURE

- **Libs folder:** Contains JARs required for development.
- **Doc folder:** Includes the API guide and reference documentation.
- **Sample folder:** Provides sample projects such as Android Demo and Java Demo, along with source code.
- **README file:** Contains integration details.

3.2.2 CONFIGURE LIBRARY DEPENDENCIES

3.2.2.1 Java Setup

1. Copy the libraries from the java/common_lib directory in the core folder to your project's libs folder.
2. Copy the libraries from the java/common_lib directory in the plugin folder to your project's libs folder.
3. **Linux:** Set up the environment and install the required Linux driver from the linux folder.
4. **Windows:** Set up the environment and install the required Windows driver from the windows folder.

3.2.2.2 Android Setup

1. Copy the libraries from the android/common_lib directory in the core folder to your project's libs folder.
2. Copy the libraries from the android/common_lib directory in the plugin folder to your project's libs folder.
3. Add the required permissions in the **AndroidManifest.xml**. Ensure you include all necessary permissions based on your SDK's requirements.

```

<!-- TCP/SSL/HTTP/HTTPS -->
<uses-permission android:name="android.permission.INTERNET" />

<!-- USB -->
<uses-feature android:name="android.hardware.usb.host" />

<!-- Bluetooth -->
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<!-- dangerous level permission, targetSdkVersion greater than or equal to 23 -->
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<!-- dangerous level permission, targetSdkVersion greater than or equal to 31 -->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"/>

<!-- Update resource, select file from external storage -->
<!-- dangerous level permission, targetSdkVersion greater than or equal to 23 -->
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>

<!-- AIDL, targetSdkVersion greater than or equal to 30 -->
<queries>
  <intent>
    <action android:name="com.pax.us.std.poslink.aidl" />
  </intent>
</queries>

```

3.2.3 SETTING UP LOGS

```
// Create a LogSetting object to configure logging
LogSetting logSetting = new LogSetting();

// Enable logging
logSetting.setEnable(true);

// Set the log level to DEBUG for detailed logs
logSetting.setLevel(LogSetting.LogLevel.DEBUG);

// Set log retention to 30 days
logSetting.setDays(30);

// Specify the log file name as "POSLog"
logSetting.setFileName("POSLog");

// Apply the log settings to the POSLinkSemi instance
POSLinkSemi.getInstance().setLogSetting(logSetting);
```

3.2.4 CONFIGURING COMMUNICATION SETTINGS

```
// Create a TcpSetting object to configure TCP communication
TcpSetting tcpSetting = new TcpSetting();

// Set the IP address of the target device
tcpSetting.setIp("192.168.1.1");

// Set the communication port number
tcpSetting.setPort("10009");

// Set the communication timeout (not the entire process timeout) in milliseconds
tcpSetting.setTimeout(60000);
```

3.2.4.1 USB Permission (Android)

If you're using `UsbSetting` on an Android device, you need to request USB permission first. You can achieve this using the following code:

```
// Request USB permission if needed
USBUtil.requestUSBPermissionIfNeed(context);
```

3.2.4.2 Serial Port Setup

For **UartSetting**, the serial port value is represented as "COMx" in Windows.
For **Linux**, the serial port value should be `/dev/ttyx`.

For **Android devices**, you can retrieve the serial port parameter dynamically by using the following method:

```
// Create a list to hold the available devices
List<String> devices = new ArrayList<>();

// Initialize SerialPortFinderPax to find serial devices
SerialPortFinderPax serialPortFinderPax = new SerialPortFinderPax();

// Get all the device paths
String[] devicesPathList = serialPortFinderPax.getAllDevicesPath();

// Iterate over the device paths and filter the ones starting with "/dev/tty"
for (String path : devicesPathList) {
    // Add paths that represent serial devices to the devices list
    if (path.startsWith("/dev/tty")) {
        devices.add(path);
    }
}
```

3.2.5 PERFORMING A TRANSACTION

```
// Get an instance of POSLinkSemi and retrieve the terminal using the given
// context and TCP settings
Terminal terminal = POSLinkSemi.getInstance().getTerminal(context, tcpSetting);

// Create a new credit transaction request
DoCreditReq doCreditReq = new DoCreditReq();

// Set the transaction type to SALE
doCreditReq.setTransactionType(TransactionType.SALE);

// Create an amount request object and set the transaction amount to "100"
AmountReq amountReq = new AmountReq();
amountReq.setTransactionAmount("100");

// Attach the amount request to the credit transaction request
doCreditReq.setAmountInformation(amountReq);

// Create a trace request object to include an ECR reference number
TraceReq traceReq = new TraceReq();
traceReq.setEcrRefNumber("1");

// Attach the trace request to the credit transaction request
doCreditReq.setTraceInformation(traceReq);

// Execute the credit transaction and store the result
ExecutionResult<DoCreditRsp> executionResult =
terminal.getTransaction().doCredit(doCreditReq);

// Check if the transaction was successful
if (executionResult.isSuccessful()) {
    // Retrieve the transaction response
    DoCreditRsp response = executionResult.response();
}
```

Note: The Terminal interface class contains three methods (getBatch(), getReport() and getTransaction()). When using the Android SDK, these methods must be executed on a worker thread rather than the Main Thread to avoid errors.

3.3 iOS/MacOS POSLink

3.3.1 FOLDER STRUCTURE

- **Doc folder:** Contains the API guide, reference documentation, and configuration instructions.
- **iOS folder:** Includes a sample project with source code and iOS library files.
- **MacOS folder:** Includes a sample project with source code and macOS library files.
- **README file:** Contains integration details.

3.3.2 CONFIGURE LIBRARY DEPENDENCIES

Copy the **POSLinkSemiIntegrationSDK** and **POSLinkAdminSDK** directories into your project.

3.3.3 SETTING UP LOGS

Note: The SDK will not save the log configuration information locally, the code must be used to set the log at the start of the application.

```
// Initialize a PLLogSetting object to configure logging
PLLogSetting *pllogSetting = [[PLLogSetting alloc] init];

// Set the log level based on the SDK Logger Level
pllogSetting.level = sdkLoggerLevel;

// Specify the directory name for log storage
pllogSetting.dirName = logSetting.fileName;

// Set the number of days to retain log files
pllogSetting.keepDays = [logSetting.days intValue];

// Apply the Log settings to the POSLinkSemi instance
[[POSLinkSemi getInstance] setLogSetting:pllogSetting];
```

3.3.4 CONFIGURING COMMUNICATION SETTINGS

```
// Initialize a PLCommSetting object to configure communication settings
PLCommSetting *commSetting = [[PLCommSetting alloc] init];

// Set the communication type to HTTPS
commSetting.commType = CommTypeHTTPS;

// Set the timeout duration (in milliseconds) for communication
commSetting.timeout = @"60000";

// Specify the destination IP address for the connection
commSetting.destIP = @"192.168.1.1";

// Specify the destination port number for the connection
commSetting.destPort = @"10009";
```

3.3.5 PERFORMING A TRANSACTION

```
// Get an instance of PLSEmiTerminal
PLSEmiTerminal *terminal = [[POSLinkSemi getInstance]
getTerminalWithCommSetting:[Manager sharedInstance].commSetting];

// Register the callback of report status
[terminal registReportStatus:^(int value) {
    NSLog(@"Terminal reports status information--%d",value);
}];

// Get an instance of the PLTransaction
PLTransaction *transaction = [terminal getTransaction];

// Create T00 request and set parameters.
PLDoCreditReq *request = [[PLDoCreditReq alloc] init];
request.transactionType = TransTypeSale;
request.amountInformation = [[PLAmountReq alloc] init];
request.amountInformation.transactionAmount = @"100";

// Request and receive response data through Block
[transaction doCreditWithRequest:request completion:^(PLDoCreditRsp * _Nonnull
response, PLExecutionResult * _Nonnull result) {
    [[POSLinkSemi getInstance] removeTerminal:terminal];
}];
```

4. Communication Setting Setup

4.1 UART

For Windows and Linux only. The PAX USB driver (available on [QUAIL resource](#)) must be installed to simulate a USB port as a serial port. POSLink will recognize the USB connection as a serial connection. Use the UART communication type when connecting via a serial or USB cable.

```
// Create a UartSetting object to configure UART communication
UartSetting uartSetting = new UartSetting();

// Set the serial port to "COM4" (port used for UART communication)
uartSetting.setSerialPort("COM4");

// Set the baud rate to 115200 (data transmission speed)
uartSetting.setBaudRate("115200");

// Set the timeout to 30 seconds (30000 milliseconds) for communication
uartSetting.setTimeout(30000);
```

4.2 TCP

```
// Create a TcpSetting object to configure TCP communication
TcpSetting tcpSetting = new TcpSetting();

// Set the IP address of the target device
tcpSetting.setIp("192.168.1.1");

// Set the communication port number
tcpSetting.setPort("10009");

// Set the communication timeout in milliseconds
tcpSetting.setTimeout(60000);
```

4.3 HTTP/HTTPS

HTTPs requires SSL Certificates. See [Section 6 SSL Communication Setup](#).

```
// Create an HttpSetting object to configure HTTP communication
HttpSetting httpSetting = new HttpSetting();

// Set the IP address of the target device
httpSetting.setIp("192.168.1.1");

// Set the communication port number
httpSetting.setPort("10009");

// Set the communication timeout in milliseconds
httpSetting.setTimeout(60000);
```

4.4 SSL

Requires SSL Certificates. See [Section 6 SSL Communication Setup](#).

```
// Create a SslSetting object to configure SSL communication
SslSetting sslSetting = new SslSetting();

// Set the IP address of the target device
sslSetting.setIp("192.168.1.1");

// Set the communication port number
sslSetting.setPort("10009");

// Set the communication timeout in milliseconds
sslSetting.setTimeout(60000);
```

4.5 Bluetooth

On Android devices, when using Android POSLink, Bluetooth can be selected as the communication type (CommType).

```
// Create a BluetoothSetting object to configure Bluetooth communication
BluetoothSetting bluetoothSetting = new BluetoothSetting();

// Set the MAC address of the target Bluetooth device
bluetoothSetting.setMacAddr("12:34:56:78");

// Set the communication timeout in milliseconds
bluetoothSetting.setTimeout(60000);
```

4.6 USB

```
// Create a UsbSetting object to configure USB communication
UsbSetting usbSetting = new UsbSetting();

// Set the USB device name or identifier
usbSetting.setDeviceName("/dev/bus/usb/001/016");

// Set the communication timeout in milliseconds
usbSetting.setTimeout(60000);
```

4.7 AIDL

```
// Create an AidlSetting object to configure AIDL communication
AidlSetting aidlSetting = new AidlSetting();
```

5. Pax Terminal Setup

Note: All terminals must have a PAX payment application installed. An application can be installed from PAXSTORE. Please see [PAXSTORE HOW TO PUSH AN APPLICATION QRG](#) on how to push an application.

5.1 Configure ECR Communication Settings via PAXSTORE

1. Login to PAXSTORE Administrator Center of the selected Marketplace.
2. Select the appropriate reseller from the Terminal Management menu.
3. Search for the terminal by its serial number, name, or terminal ID.
4. Access the Push Task Configuration page by selecting “+PUSH APP” button to push an application to the terminal.
5. Select the communication tab.
6. To set communication to Ethernet, set Communication Type parameter to “Ethernet” in the Communication Between ECR/POS and Pax Terminal Group. Then set the Protocol to TCP, HTTP, HTTPS or SSL.
7. To set communication to USB, set Communication Type parameter to “USB” in the Communication Between ECR/POS and Pax Terminal Group.

Edit Parameter

Please fill in the form of the parameter file. System will use the parameter values in the last success push history by default, or you can click [HERE](#) to reset the values to the default values in the parameter template.

INDUSTRY

EDC

RECEIPT

TIP

MISC

HPSPTC

COMMUNICATION

CARD TYPE

BIN FILE

EMV

BROADPOS

PATT

5.2 Configure ECR Communication Settings via Terminal settings

5.2.1 PAX ANDROID TERMINALS

To access the Settings menu, tap the four corners of the screen in the following sequence: **Top Left → Top Right → Bottom Right → Bottom Left**.

The default password is "1" or today's date in **MMDDYYYY** format.

5.2.1.1 AIDL

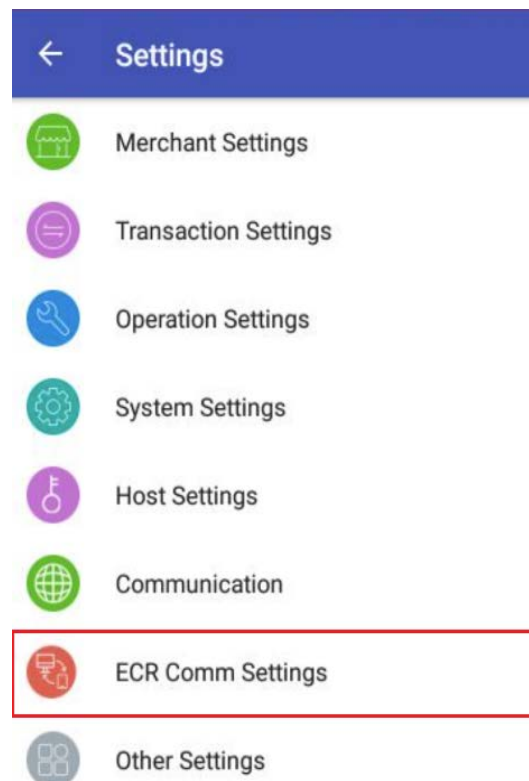
Android-based devices currently support both AIDL and IP communication types. **AIDL is the recommended communication type for Android developers running their POS app directly on the PAX**

terminal, providing an all-in-one solution. AIDL offers several advantages over other communication types:

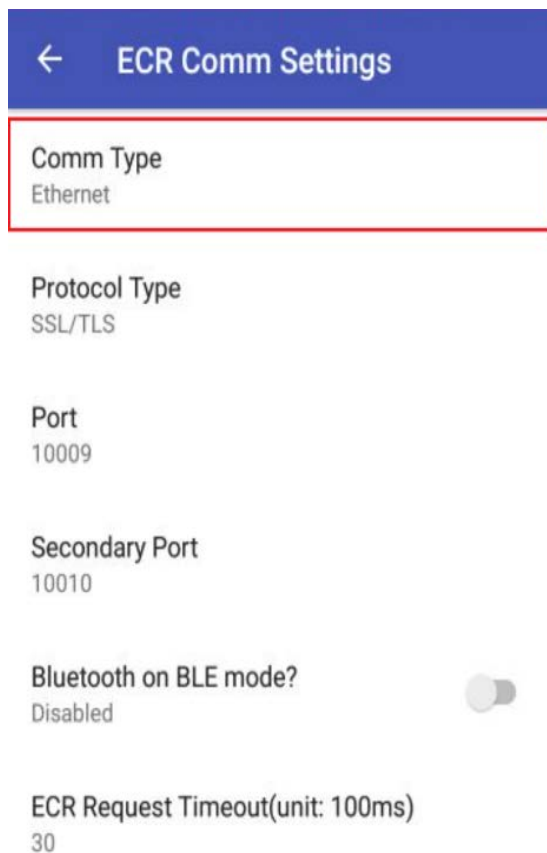
1. It can wake up BroadPOS if it is not already running.
2. AIDL commands from the POS to BroadPOS bypass the currently set ECR Comm Setting within BroadPOS.
3. It generally delivers faster performance due to minimal latency and lower overhead.
4. For external POS setups, IP communication remains an option.

5.2.1.2 USB

1. Access Settings Menu.
2. Select ECR Comm Settings.

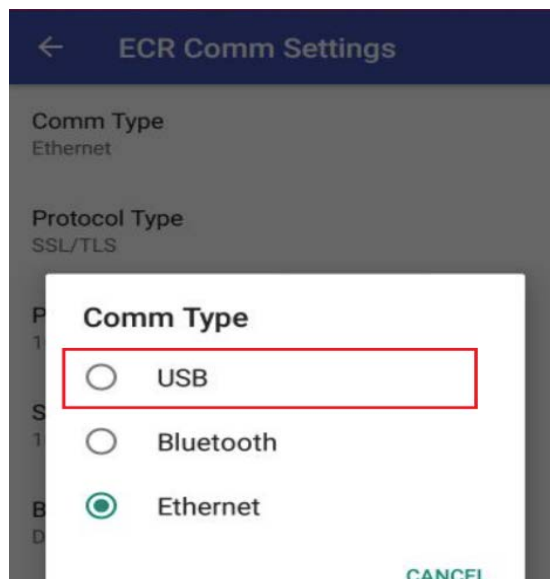


3. Go to Comm Type.



The screenshot shows the 'ECR Comm Settings' screen. The 'Comm Type' is set to 'Ethernet' and is highlighted with a red box. Other settings visible include 'Protocol Type' set to 'SSL/TLS', 'Port' set to '10009', 'Secondary Port' set to '10010', 'Bluetooth on BLE mode?' set to 'Disabled' with a toggle switch, and 'ECR Request Timeout(unit: 100ms)' set to '30'.

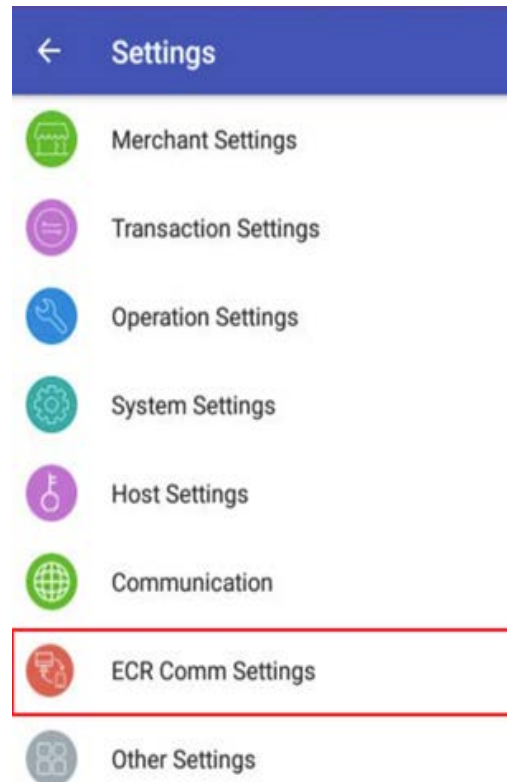
4. Select USB.



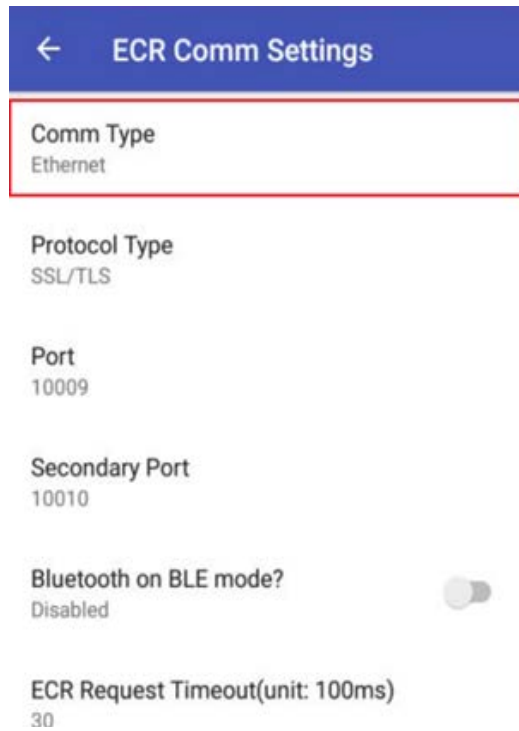
The screenshot shows the 'ECR Comm Settings' screen with a 'Comm Type' selection dialog box open. The dialog box has a title 'Comm Type' and three options: 'USB' (highlighted with a red box), 'Bluetooth', and 'Ethernet' (which is currently selected with a green dot). A 'CANCEL' button is visible at the bottom right of the dialog box.

5 .2.1.3 Bluetooth

1. Access Settings Menu.
2. Select ECR Comm Settings.



3. Go to Comm Type.



← ECR Comm Settings

Comm Type
Ethernet

Protocol Type
SSL/TLS

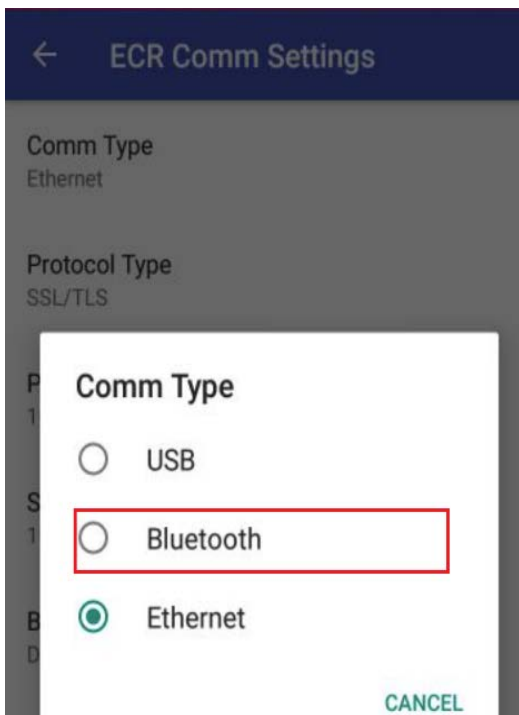
Port
10009

Secondary Port
10010

Bluetooth on BLE mode?
Disabled

ECR Request Timeout(unit: 100ms)
30

4. Select Bluetooth.



← ECR Comm Settings

Comm Type
Ethernet

Protocol Type
SSL/TLS

Comm Type

☐ USB

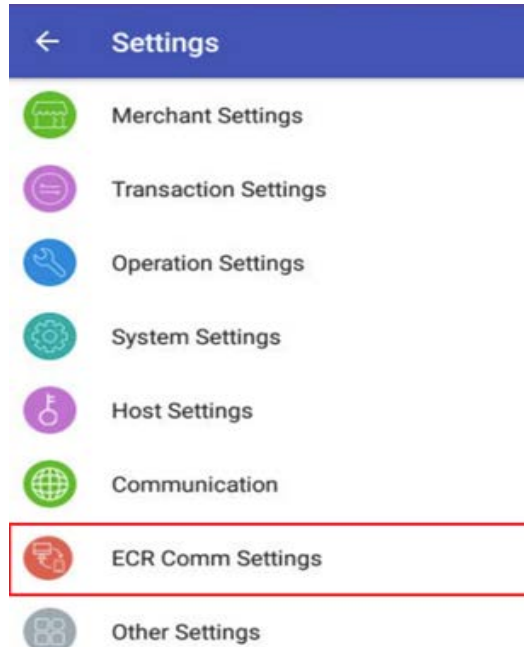
☐ Bluetooth

☒ Ethernet

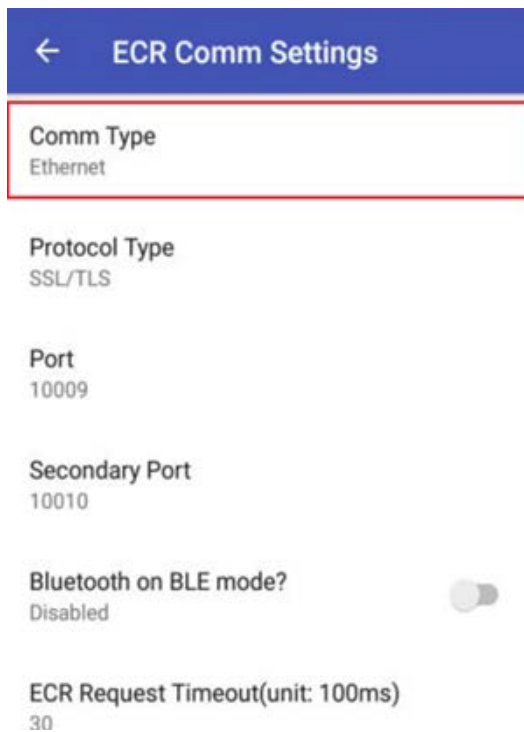
CANCEL

5 .2.1.4 Ethernet - TCP/IP, HTTP, HTTPS, SSL

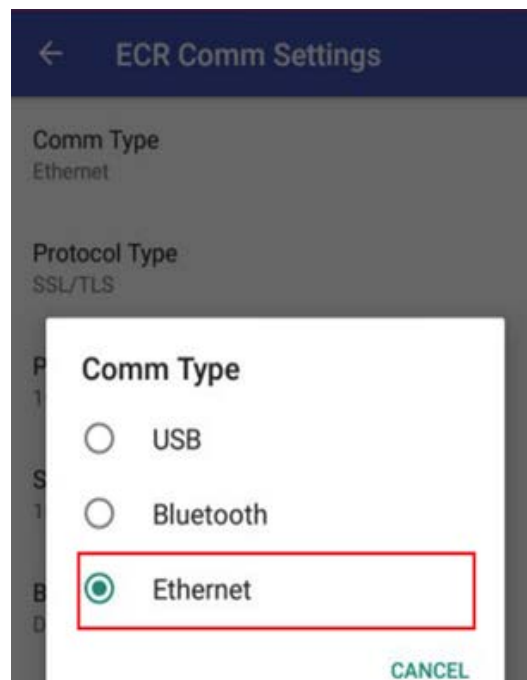
1. Access Settings Menu.
2. Select ECR Comm Settings.



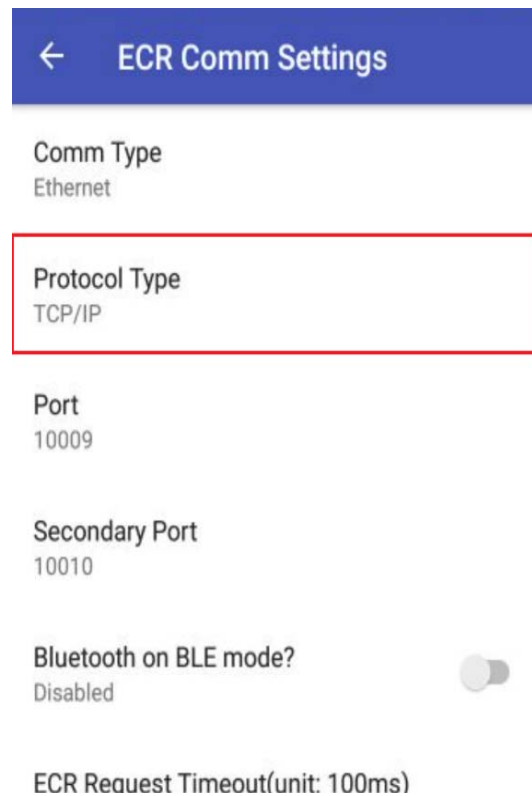
3. Go to Comm Type.



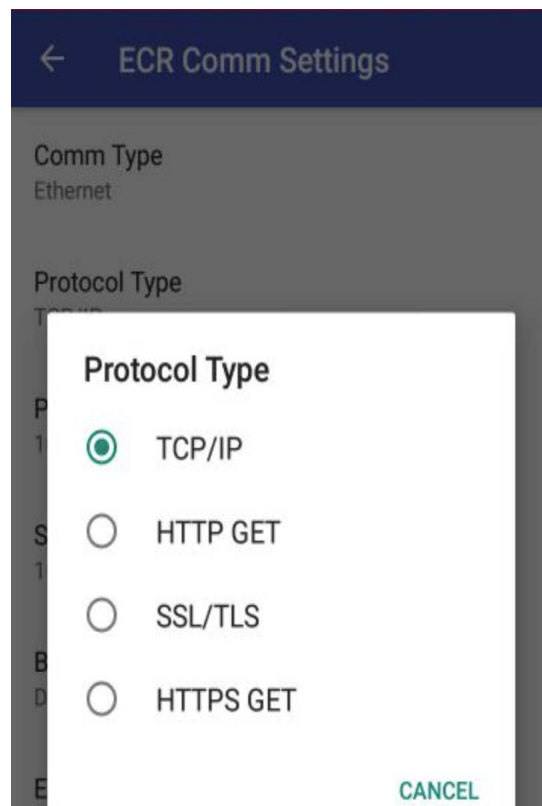
4. Select Ethernet.



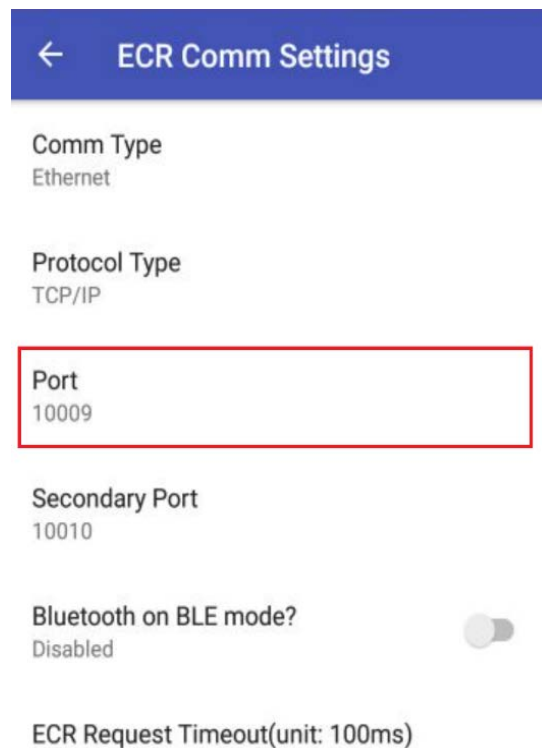
5. Go to Protocol Type.



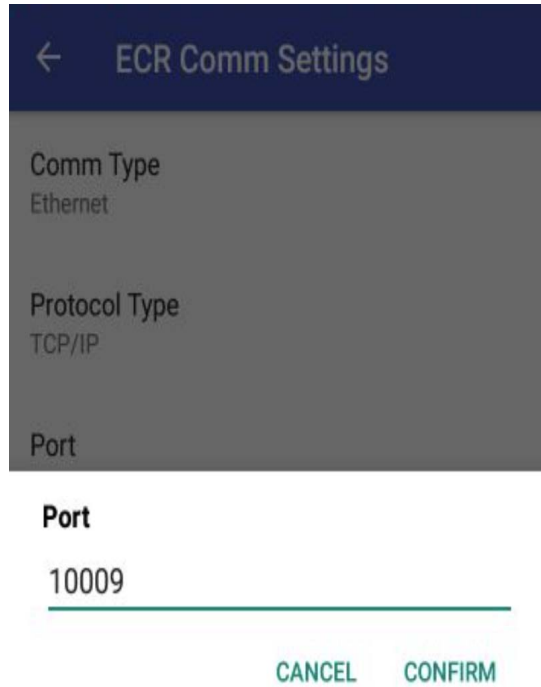
6. Select TCP/IP, HTTP, HTTPS or SSL.



7. Go to Port.



8. Enter Communication Port Number. Default port is 10009.

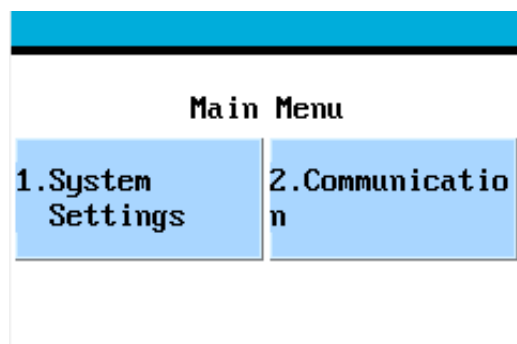


5.2.2 TRADITIONAL TERMINALS

PAX traditional terminals use different methods depending on the terminal model to access the settings menu.

- Physical keys - S920

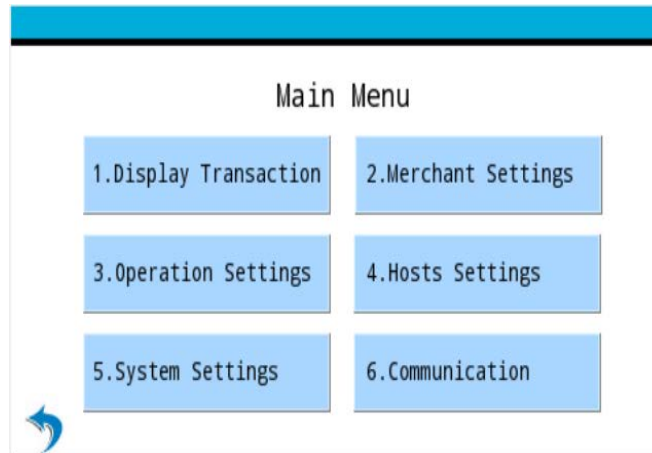
- Press and hold the Enter Key  and then press the Num 1 Key  at the same time to access Main Menu.
- Enter Password. Default Password is today's date (format: mmddyyyy).
- Scroll down using the on screen down arrow .
- Select Communication.



5. Enter same password as above.

• Touch Screen - Q20

1. The Settings menu shall accessible by tapping four corners of the screen in the following order:
Top Left -> Top Right -> Bottom Right -> Bottom Left.
2. Enter Password. Default Password is 1 or today's date (format: mmddyyyyy).
3. Select Communication.

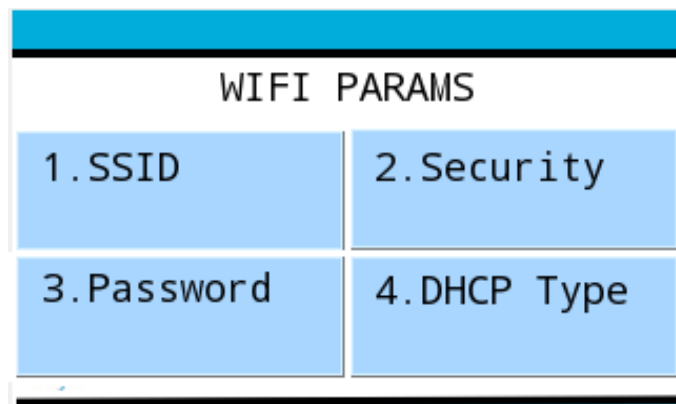


• Functional Keys - Q25

1. Press the right function key.
2. Enter Password. Default Password is 1 or today's date (format: mmddyyyyy).

5 .2.2.1 ETHERNET-TCP/IP, HTTP, HTTPS, SSL

1. In terminal settings access communication menu.
2. Scroll down using the on screen down arrow.
3. Select Wi-Fi Parameters
4. Select SSID. Or if previously configured, scroll down using the on screen down.



5. Choose the SSID of the WIFI router.
6. Select Password and enter Access Point Preshared Key.

7. Press Cancel Key .

8. Scroll down twice using the Down Arrow Key .

9. Select ECR Comm. Type.

Communication Options	
1.Receive Timeout	2.Wireless Parameters
3.WiFi Parameters	4.ECR Comm. Type

10. Select Ethernet.

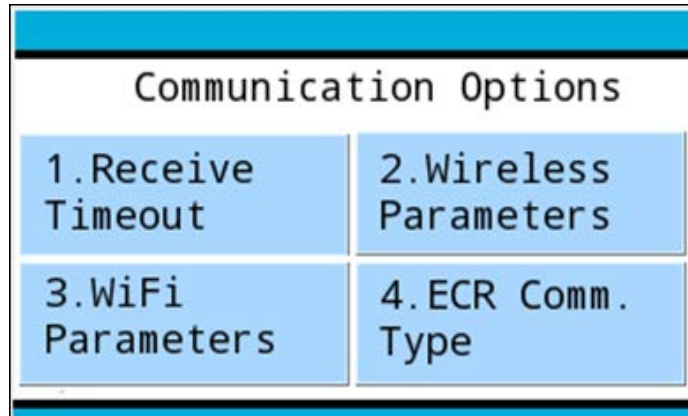
ECR Communication Type: 1	
1.USB	2.Ethernet

11. Enter Communication Port Number. Default port is 10009.

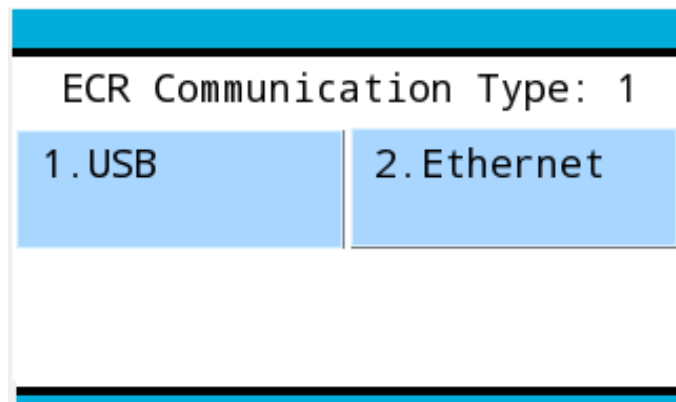
COMM. PORT
10009

5 .2.2.2 USB

1. Access Communication Menu.
2. Scroll down twice using the on screen down arrow .
3. Select ECR Comm. Type.



4. Select USB.



6. HTTPS/SSL Communication Setup Requirement

6.1 PEM Files and Requirements

PAX Terminals support SSL communication for the POS system. The terminal needs to have the following SSL PEM certificate files loaded:

6.1.1 SERVERKEY.PEM - PRIVATE KEY

Supported Formats:

1. PKCS#1 format:

- enclosed between "-----BEGIN RSA PRIVATE KEY-----" and "END RSA PRIVATE KEY-----" **[[Recommended]]**

2. PKCS#8 format:

- enclosed between "-----BEGIN PRIVATE KEY-----" and "-----END PRIVATE KEY-----" Limitation: browser must be later than 20171103 ssl/https server will not be started successfully for other formats.

3. Converting Between Formats:

- Convert PKCS#1 to PKCS#8**
 - \$ openssl pkcs8 -topk8 -inform PEM -in pkcs1.pem -outform pem -nocrypt -out pkcs8.pem
- Convert PKCS#8 to PKCS#1**
 - \$ openssl rsa -in pkcs8.pem -out pkcs1.pem

6.1.2 SERVERCA.PEM, SERVERCERT.PEM

serverca.pem – used by terminal to verify certificate from client, usually with empty text since current SSL server is built on PAX device is not verifying certificate from client.

servercert.pem – append all certificates defined in customer provided ca.pem to this file using format below. The first certificate should be the local certificate used by the terminal and the others are CA certificates.

Supported Formats:

- enclosed between "-----BEGIN CERTIFICATE-----" and "-----END CERTIFICATE-----"
- enclosed between "-----BEGIN X509 CERTIFICATE-----" and "-----END X509 CERTIFICATE-----"
- enclosed between "-----BEGIN TRUSTED CERTIFICATE-----" and "-----END TRUSTED CERTIFICATE-----"

Note: ssl/https server will not be started successfully for other formats.

6.1.3 VERIFYING THAT THE PRIVATE KEY MATCHES A CERTIFICATE

```
$ openssl x509 -noout -modulus -in servercert.pem | openssl md5
```

```
$ openssl rsa -noout -modulus -in serverkey.pem | openssl md5
```

Note: MD5 value should be same, or otherwise the serverkey.pem will not match servercert.pem

6.1.4 READ CERTIFICATE INFORMATION

```
$ openssl x509 -in servercert.pem -noout -text
```

1. Issuer: who sign this certification, usually is CA
2. Subject : who own this certification
3. CA: True: this is a CA; False: this is not a CA

Below is result for command [openssl x509 -in servercert.pem -noout -text]

```
=====
Certificate:
    Data:
        Version: 3 (0x2) Serial Number:
            48:e0:be:c3:86:9a:09:53:1a:d3:de:3d:7d:28:24:5c
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA
        Limited, CN=COMODO RSA Domain Validation Secure Server CA
        Validity
            Not Before: Jul 24 00:00:00 2015 GMT
            Not After : Jul 23 23:59:59 2018 GMT
        Subject: OU=Domain Control Validated, OU=PositiveSSL
        Wildcard, CN=*.econduitapps.com
    Subject Public Key Info:
        Public Key Algorithm: rsaEncryption Public-Key: (2048 bit)
        Modulus: 00:dd:24:76:09:67:b4:3c:83:ad:1e:52:1b:aa:50:
            af:96:79:80:cb:d8:97:53:76:bc:b2:4a:ca:e4:3d:
            54:de:0e:5e:94:2b:6f:00:6e:4c:0e:94:ab:06:8e:
            48:bb:39:33:69:2d:fe:f6:73:4b:6a:e7:c3:ed:de:
            28:fc:e1:5a:14:f3:4f:8b:60:6d:79:7f:bc:0c:07:
            e5:7c:54:52:6f:28:6c:85:9d:fc:bd:69:30:37:78:
            84:79:88:0d:ec:94:9b:19:22:e7:bb:72:c7:ea:ee:
            e8:fd:2c:6d:23:76:b7:52:31:75:74:ed:f7:4b:41:
            8f:19:14:14:62:fc:65:a8:7a:87:e1:99:a1:d5:69:
            3d:07:83:6a:dd:e6:97:79:d2:c1:30:80:d1:fb:05:
            d6:b0:be:ad:63:fd:0f:77:7c:4d:35:b9:23:83:b4:
            c4:d8:32:59:04:d3:72:42:49:6a:5f:8c:d5:b9:17:
            61:6c:24:f2:b4:56:4f:59:89:c2:df:eb:ec:55:d1:
```

b6:7c:40:f5:a8:53:8e:e6:73:d4:23:2e:d7:05:7b:
de:b6:74:bb:7b:55:c2:d9:65:e7:68:55:06:67:96:
d3:87:41:26:ce:94:bb:17:e5:67:f6:3a:83:77:8c:
26:4d:ba:b6:26:90:e1:f6:35:22:2a:1d:49:34:b7:
74:5d

Exponent: 65537 (0x10001) X509v3 extensions:

X509v3 Authority Key Identifier:

keyid:90:AF:6A:3A:94:5A:0B:D8:90:EA:12:56:73:DF:43:B4:3A:28:DA:E7

X509v3 Subject Key Identifier:

B7:49:7C:01:3B:06:68:56:48:BE:DC:DF:B1:AD:FD:D0:4B:6E:08:44

X509v3 Key Usage: critical

Digital Signature, Key Encipherment

X509v3 Basic Constraints: critical CA:FALSE

X509v3 Extended Key Usage:

TLS Web Server Authentication, TLS Web Client Authentication

X509v3 Certificate Policies:

Policy: 1.3.6.1.4.1.6449.1.2.2.7

CPS: <https://secure.comodo.com/CPS> Policy: 2.23.140.1.2.1

X509v3 CRL Distribution Points: Full Name:

URI:<http://crl.comodoca.com/COMODORSADomainValidationSecureServerCA.crl>

Authority Information Access:

CA Issuers - URI:<http://crt.comodoca.com/COMODORSADomainValidationSecureServerCA.crt>

OCSP - URI:<http://ocsp.comodoca.com>

X509v3 Subject Alternative Name: DNS:*.econduitapps.com, DNS:econduitapps.com

Signature Algorithm: sha256WithRSAEncryption

00:dc:62:da:42:52:15:8a:37:c3:21:3d:12:f8:58:bc:b3:0c:
9e:01:8a:cb:ff:6c:33:f8:51:cf:6d:3c:41:c6:a1:7d:ca:91:
7b:a3:6e:f5:d4:d1:8f:41:64:d8:1b:63:78:ac:a0:11:c0:69:
65:3f:2a:09:60:20:40:fe:3b:68:77:53:af:ca:e7:fb:ac:35:
c3:31:73:fe:de:55:31:82:94:ed:a3:2f:f8:64:91:e7:fc:83:
91:63:08:47:0f:26:22:b3:d4:f5:fc:5a:48:c0:c5:5f:7c:8e:
b1:c2:52:40:57:e2:4d:ec:0f:35:6a:13:56:6b:1e:2b:a1:f2:
9b:b5:4b:ee:0d:8d:77:0d:f3:ce:1b:62:bc:4f:cc:56:c9:f4:
dc:81:0c:ff:16:75:08:f8:23:87:ab:df:7a:11:97:99:4f:90:
2c:8a:69:62:0e:10:46:51:b6:04:b1:6e:99:a5:69:6b:f0:e3:
4a:d1:9f:3d:fe:15:af:13:25:b5:01:08:c1:b6:59:b4:ad:86:
d2:cf:1b:3b:c2:e6:e1:56:0f:1c:74:b7:7b:ae:f8:ea:4d:08:
1d:3a:06:ae:b7:b2:07:7a:5a:79:78:1f:cb:40:0c:ed:8f:a0:
92:41:14:6c:9c:5f:f2:3c:1c:95:07:f9:a8:06:a1:0a:b2:78:

4e:25:90:73

NOTE: Key with password is not supported, you should either get a key without password from CA or convert the key with password to a key without password by using openssl rsa command.

Example:

```
$openssl rsa -in rsa_aes_private.key -passin pass:111111 -out rsa_private.key
```

6.2 Load PEM Files to PAX Terminal

The certificate files can be uploaded to the terminal via BroadPOS PAXSTORE.

1. In the Edit Parameter Page, select the Communication Tab.
2. Under POS System Feature (Ethernet Only), there will be upload fields.
3. Browse and upload the corresponding certificate files to respective fields.

POS SYSTEM FEATURE(ETHERNET ONLY)

Access Control Allow Origin for HTTP/HTTPS

Enabled

PEMS FOR SSL/HTTPS PROTOCOL

Private Key

SERVERKEY.PEM

Trusted Cert. List

SERVERCA.PEM

Server Certificate

SERVERCERT.PEM

6.3 Generating a self-signed certificate

1. Make demoCA directory:
 - `$ mkdir -p ./demoCA/{private,newcerts}`
 - `$ touch ./demoCA/index.txt`
 - `$ echo 01 > ./demoCA/serial`
2. Create a self-signed CA:
 - `$ openssl req -new -x509 -extensions v3_ca -sha256 -days 3650 -key ./demoCA/private/cakey.pem`
3. Generate ca private key, RSA 2048 bit:
 - `$ openssl genrsa -out ./demoCA/private/cakey.pem 2048`
4. Generate server private key, RSA 2048 bit:
 - `$ openssl genrsa -out serverkey.pem 2048`
5. Use server private key to create a Certificate Signing Request (CSR) file:
 - `$ openssl req -new -sha256 -days 3650 -key serverkey.pem -out serverReq.pem`
6. Sign the server's CSR with demoCA, make sure [countryName, stateOrProvinceName, organizationName] match during CA creation:
 - `$ openssl ca -in serverReq.pem -out servercert.pem`
7. Copy serverca.pem to current directory
 - `$ cp ./demoCA/cacert.pem serverca.pem`

6.4 Import Self-Signed CA to Trusted Root Certification Authorities (Windows Only)

1. Win -> mmc.exe
2. Select File and click **Add/Remove Snap-in...**
3. Select the **Certificates** snap-in, and click **Add**
4. Select **Computer account**, and click **Next**
5. Select **Local computer**, and click **Finish**
6. Click **OK**
7. In the left pane, expand **Certificates (Local Computer)**
8. Expand **Trusted Root Certification Authorities**, and click **Certificate**
9. Right-click on the newly created certificate, select **All Tasks**, and click **Import...**
10. The Certificate Export Wizard will open. Click **Next** to continue.
11. Specify a file name and click **Next**.
12. Click **Next**
13. Click **Finish**

PAX Customer Support

For questions or help with the terminal installation please contact your service provider or PAX customer support.

PAX Technology Inc.
8775 Baypine Road
Jacksonville, FL 32256

support@pax.us
(877) 859-0099
www.pax.us

This document is provided for informational purposes only. All features and specifications are subject to change without notice. The PAX name and PAX logo are registered trademarks of PAX Corporation and/or its affiliates. Other names may be trademarks of their respective owners. Copyright © 2024, PAX Technology Limited, all rights reserved.

****PAX Technology, Inc. is not responsible for the content, quality, accuracy, or completeness of any information or materials contained on these pages. PAX Technology, Inc. does not endorse any content, viewpoints, products, or services contained on these pages and shall not be held liable for any losses caused by reliance on the accuracy, reliability, or timeliness of such information. Any person or entity that relies on any information obtained from these pages does so at his/her own risk.***